

A IA Perfeita — Vol. 2: Códigos, Prompts & Algoritmos

Implementações completas, prompt library avançada e os algoritmos agentic que sustentam a próxima geração de sistemas autônomos

Coleção A IA Perfeita — Engenharia Agentic Sistêmica • Vol. 2 de 3

Por Nexus HUB57 · Ecossistema MMN_IA

MMN_IA • 2026

Sobre este ebook

O Vol. 2 é o **livro de receitas** da coletânea. Onde o Vol. 1 tratou de **padrões e arquiteturas** em alto nível, este volume mergulha em **códigos reais, prompt templates prontos para uso, e os algoritmos agentic mais importantes** em produção em 2026. É referência técnica direta: copie, adapte, envie para produção.

Sumário

- 1. Filosofia do código agentic • 2. Algoritmo 1 — Plan-and-Execute completo • 3. Algoritmo 2 — Reflexion (auto-crítica iterativa) • 4. Algoritmo 3 — Tree of Thoughts • 5. Algoritmo 4 — Self-Consistency Sampling • 6. Algoritmo 5 — ReWOO (Reasoning With tools Only) • 7. Algoritmo 6 — MRKL (Modular Reasoning, Knowledge, Language) • 8. Algoritmo 7 — BabyAGI • 9. Algoritmo 8 — AutoGPT loop • 10. Algoritmo 9 — CAMEL (Role-Playing) • 11. Algoritmo 10 — Voyager (Skill Library Growth) • 12. Algoritmo 11 — DSPy (declarative prompting) • 13. Algoritmo 12 — Constitutional AI • 14. Algoritmo 13 — RLHF simplificado • 15. Algoritmo 14 — RAG híbrido avançado • 16. Algoritmo 15 — Agent Memory (episódica + semântica) • 17. Algoritmo 16 — Self-Healing Retry com backoff exponencial • 18. Algoritmo 17 — Tool Router (decision tree) • 19. Algoritmo 18 — Critic/Refiner loop • 20. Algoritmo 19 — Agent Orchestrator (state machine) • 21. Algoritmo 20 — Streaming com cancelamento • 22. Prompt Library — Fundamentos • 23. Prompt Library — Estruturados (JSON) • 24. Prompt Library — Chain-of-Thought • 25. Prompt Library — Few-Shot • 26. Prompt Library — ReAct • 27. Prompt Library — Persona • 28. Prompt Library — Constitutional • 29. Prompt Library — RAG • 30. Prompt Library — Tools • 31.

1. FILOSOFIA DO CÓDIGO AGENTIC

O código agentic moderno se distingue do código tradicional por **três propriedades**:

1. **Estado é distribuído** — conversa, memória, tools, retrieval, reflexão.
2. **Fluxo é cíclico** — não há "fim" definitivo; há iteração até convergência ou timeout.
3. **Erro é matéria-prima** — toda falha é informação para o próximo passo.

A função do engenheiro agentic é, portanto, **menos controlar e mais habilitar**. Você não diz ao agente **o que fazer**; você constrói o **palco onde ele pode decidir**.

2. ALGORITMO 1 — PLAN-AND-EXECUTE COMPLETO

```
import asyncio
import sys
from typing import List, Dict, Any
from datetime import datetime
from pathlib import Path

# Separa o nome do agente da chamada LLM de execução (ex: "gpt-4o")
def parse_prompt(prompt: str) -> tuple[str, str]:
    """
    Retorna (nome_do_agente, prompt_formatado)
    """
    nome, prompt_formatado = prompt.split(":", 1)
    nome = nome.strip()
    prompt_formatado = prompt_formatado.strip()

    # Se não houver nome, usa o padrão
    if not nome:
        nome = "gpt-4o"

    return nome, prompt_formatado

def main():
    """
    Main function to run the Plan-and-Execute algorithm.
    """
    # 1. Definir o nome do agente e o prompt
    nome_agente, prompt_formatado = parse_prompt(sys.argv[1])

    # 2. Criar o contexto (tools, memory, etc.)
    context = {
        "tools": [],
        "memory": [],
        "history": []
    }

    # 3. Iniciar o loop de execução
    async def execute_plan():
        """
        Execute the plan and return the result.
        """
        # Aqui você implementa a lógica de execução do plano
        # Exemplo: chamar a API do OpenAI
        response = await call_openai_api(
            model=nome_agente,
            prompt=prompt_formatado,
            context=context
        )

        # 4. Atualizar o contexto com o resultado
        context["history"].append(response)

        # 5. Retornar o resultado
        return response

    # 6. Executar o plano
    result = asyncio.run(execute_plan())

    # 7. Exibir o resultado
    print(result)

if __name__ == "__main__":
    main()
```

```

    if step.get('action') == 'tool' and self._actions.get(step.get('tool')):
        tool = self._actions.get(step.get('tool'))
        model = self._model(max_tokens=1024, messages=self._role + user.get('content'))
        return tool.run(tool.get('step'), tool.get('args'))
    elif step.get('action') == 'wait':
        tool = self._actions.get(step.get('tool'))
        return {'status': 'no tool', 'step': step}
    elif not self._check_policy(tool.get('policy'), step.get('args')):
        return {'status': 'denied', 'step': step}
    try:
        result = tool.run(tool.get('step'), tool.get('args'))
        return {'status': 'ok', 'step': step, 'result': result}
    except Exception:
        return {'status': 'error', 'step': step, 'error': True}

    def run(self, goal, step=None):
        plan = self._plan(goal)
        results = []
        for step in plan:
            result = self._run(step)
            results.append(result)
        return {'plan': plan, 'results': results}

    def _check_policy(self, policy, args):
        return policy == 'always'

    def _extract_result(self, result):
        return result.get('text')

```

3. ALGORITMO 2 — REFLEXION (AUTO-CRÍTICA ITERATIVA)

```

class ReflexionAgent:
    """Gera crítica, reflete, aprende com a própria falha"""
    def __init__(self, llm, max_iter=10):
        self.llm = llm
        self.max_iter = max_iter
        self.reflections = []

```

```

def run(self, task, stop = stop):
    solution = self._initialTask()
    for i in range(1, self.steps):
        critique = self._critique(task, solution)
        if not critiqued:
            return solution
        solution = self._refine(task, solution, critique)
        self.reflections.append((i, solution))
    return solution

def _initial(self, task):
    return self._initialTask(task)

def _critique(self, task, sol):
    return self._critique(task, sol)

def _refine(self, task, sol, crit):
    return self._refine(task, sol, crit)

```

4. ALGORITMO 3 — TREE OF THOUGHTS

```

import heapq

class TreeOfThoughts:
    def __init__(self, candidates, eval_fn, rank_fn):
        self.root = None
        self.time = 0
        self.eval = eval_fn
        self.rank = rank_fn
        self.frontier = []
        self.children = []

    def solve(self, problem):
        self.root = problem
        self.frontier = [(self.root.score, self.root)]
        best = None
        for d in range(1, self.steps):
            new_frontier = []
            for node in self.frontier:
                children = self._expand(node)
                for c in children:
                    c.score = self._eval(c)
                    new_frontier.append((c.score, c))
            self.frontier = new_frontier
            best = self._select_best()
            self.frontier = new_frontier
        return best

```



```

def _evaluate(self, plan):
    # resolver = plan.resolver
    # ...

```

7. ALGORITMO 6 — MRKL

```

class MRKLSystem:
    """Router for special tools"""
    |
    |
    def __init__(self):
        self._lim = 0
        self._specialists = {}
    |
    def run(self, query):
        """Run MRKL system"""
        query = query
        _specialist = self._specialists.get(query)
        if _specialist:
            _specialist.run(query)
        else:
            _specialist = self._create_specialist(query)
            self._specialists[_specialist] = _specialist
        return self._specialists[_specialist].run(query)

```

8. ALGORITMO 7 — BabyAGI

```

class BabyAGI:
    """Loop in which it is created, evaluated, prioritized, iterated"""
    |
    def __init__(self, lim, vector_db, tools):
        self._lim = lim
        self._vector_db = vector_db
        self._tools = tools
        self._task_list = []
        self._task_obj = {}
        self._task_status = {}
        self._task_result = {}
        self._task_obj['objective'] = 'status: open'
        self._task_status['status'] = 'open'
        self._task_result['result'] = 'open'
        self._task_obj['task'] = 'task: open'
        self._task_status['status'] = 'open'
        self._task_result['result'] = 'open'
        self._task_obj['task'] = 'task: open'
        self._task_status['status'] = 'open'
        self._task_result['result'] = 'open'

```

```
def get_new_tasks(self):
    """Get new tasks"""
    new_tasks = []
    for task in self.tasks:
        if task.status == 'pending':
            new_tasks.append(task)
    return new_tasks

def create_new_tasks(self, new_tasks):
    """Create new tasks"""
    self.tasks.extend(new_tasks)

def report_tasks(self):
    """Report tasks"""
    print("Tasks: ", self.tasks)
```

9. ALGORITMO 8 – AutoGPT LOOP

Health Condition	Percentage of Respondents
Allergies	15%
Asthma	35%
Chronic pain	12%
Diabetes	18%
Heart disease	10%
Hypertension	22%
Obesity	28%
Seizures	14%
Stomach issues	1%
Depression	25%
Anxiety disorders	30%
Bipolar disorder	38%
Chronic fatigue	20%
Disruptive sleep	32%
Headaches	35%
Menstrual issues	35%
Migraines	10%
Post-traumatic stress disorder	45%
Self-harm	40%
Substance use	48%
Trauma	22%
Unexplained symptoms	30%
Worry	42%
Other	1%
None of the above	25%

10. ALGORITMO 9 – CAMEL (Role-Playing)

Label Type	Percentage of Respondents
Main label	100%
Small label	90%
Barcode label	80%
QR code label	70%

```

self.user = user()
self.assistant = assistant()
|
def run(self, task, rounds=10):
    history = []
    for _ in range(rounds):
        inp = self.user.generate(f"User: {task} {history[-4:] if history else ''}")
        history.append(inp)
        response = self.assistant.generate(f"Assistant: {response[-4:] if history else ''}")
        history.append(response)
    return history

```

11. ALGORITMO 10 — VOYAGER (Skill Library Growth)

```

class Voyager:
    """Agente que aprende skills e cresce uma biblioteca de código"""
    |
    def __init__(self, llm, sandbox):
        self.llm = llm
        self.sandbox = sandbox
        self.skills = []
    |
    def run(self, user_prompt):
        while True:
            skill_needed = self.llm.generate(f"User: {user_prompt} \nSkills: {self.skills[-4:] if self.skills else ''}")
            if skill_needed:
                self.sandbox.run(self.task(skill_needed))
            else:
                new_skill_code = self.llm.generate(f"Escreva um código Python para {skill_needed}")
                self.skills.append(skill_needed + "\n" + new_skill_code)
                self.sandbox.run(new_skill_code)
            if None in skill_needed.lower().split():

```

12. ALGORITMO 11 — DSPy (DECLARATIVE PROMPTING)

```

import dspy
|

```

```

class Summarizer(DSPy.Module):
    def __init__(self):
        self.summary_size = dspy.ChatGPT(prompt="document = summary")
    def forward(self, document):
        return self.summary_size(document=document)
    def __call__(self, document):
        return self.forward(document=document)
    def __repr__(self):
        return f"Summarizer (prompt={self.summary_size.prompt})"

```

13. ALGORITMO 12 — CONSTITUTIONAL AI

```

class ConstitutionalAI:
    def __init__(self, principles):
        self.principles = principles
    def __call__(self, prompt):
        response = self.generate(prompt=prompt)
        critique = self.generate(prompt=prompt, response=response)
        principles = self.principles
        response = self.generate(prompt=prompt, response=response, critique=critique)
        return response

```

14. ALGORITMO 13 — RLHF SIMPLIFICADO

```

class SimpleRLHF:
    def __init__(self, policy, reward_model, ref_policy):
        self.policy = policy
        self.reward = reward_model
        self.ref = ref_policy
    def __call__(self, prompt, response):

```

```

    r = self.reward_function(prompt, response)
    R = self.get_reward(self.prompt + self.response, prompt, response)
    return R

```

15. ALGORITMO 14 — RAG HÍBRIDO AVANÇADO

```

class HybridRAG:
    def __init__(self, bm25_indexer, bm25_searcher, reranker):
        self.bm25 = bm25_indexer
        self.bm25_searcher = bm25_searcher
        self.reranker = reranker

    def retrieve(self, query, k=20, top_k=5):
        bm25_hits = self.bm25.search(query, k)
        docs = [bm25_indexer.doc(bm25_hit.docid) for bm25_hit in bm25_hits]
        # Reciprocal Rank Fusion
        fused_scores = self.bm25_searcher.rerank(docs)
        return self.reranker.rerank(fused_scores, top_k)

    def run(self, query, k=60):
        scores = []
        docs = []
        scores_docs = scores_docs = self.bm25_searcher.rerank(docs)
        for rank, doc in enumerate(docs):
            scores_docs[rank] = scores_docs[rank] + (1 / (rank + 1))
        return sorted(scores_docs, reverse=True)[:k]

```

16. ALGORITMO 15 — AGENT MEMORY (EPISÓDICA + SEMÂNTICA)

```

class AgentMemory:
    def __init__(self, episodic_indexer, semantic_indexer, importance_threshold):
        self.episodic = episodic_indexer
        self.semantic = semantic_indexer
        self.threshold = importance_threshold

    def store(self, event):
        importance = self.score_event(event)
        if importance > self.threshold:
            self.episodic.store(event)
            self.semantic.store(event)

```

Category	Percentage
Self-reported consumption	70.0%
Self-reported consumption	85.0%
Police in control	50.0%
Self-reported consumption	75.0%
Police in control	50.0%
Self-reported consumption	75.0%
Police in control	50.0%
Self-reported consumption	75.0%
Police in control	50.0%

17. ALGORITMO 16 — SELF-HEALING RETRY

[illegible]

18. ALGORITMO 17 — TOOL ROUTER (DECISION TREE)

[illegible]

19. ALGORITMO 18 — CRITIC/REFINER LOOP

```
class CriticRefiner:
    """Critic/Refiner Loop"""
    def __init__(self, generator):
        self.gen = generator
        self.prompt = ""
        self.threshold = 0.5
        self.max_iter = 10
    def run(self, prompt):
        out = self.gen.generate(prompt)
        for i in range(self.max_iter):
            score = self.gen.score(out)
            if score > self.threshold:
                return out
            feedback = self.gen.feedback(out)
            self.prompt = self.prompt + feedback
        return out
```

20. ALGORITMO 19 — AGENT ORCHESTRATOR (STATE MACHINE)

```
from enum import Enum
class State(Enum):
    """State Enum"""
    IDLE = "idle"
    PLANNING = "planning"
    EXECUTING = "executing"
    REFLECTING = "reflecting"
    DONE = "done"
    FAILED = "failed"
class StateMachineOrchestrator:
    """State Machine Orchestrator"""
    def __init__(self, layers):
        self.layers = layers
        self.state = State.IDLE
        self.context = {}
    def transition(self, new_state):
        self.state = new_state
        self.layers[-1].tracef("state: {new_state.value} | self.context: {self.context}")
```

```

def run(self, goal):
    self.transi.constate.planning
    self.context["plan"] = self.layers["planner"].plan(goal)
    self.transi.constate.planning
    for step in self.context["plan"]:
        if self.layers["executor"].execute(step):
            self.transi.constate.planning = False
            if not step["is_denied"]:
                self.transi.constate.done
            else:
                self.transi.constate.reflecting
    self.context["reflecting"] = self.layers["reflect"].reflect(self.context)
    self.transi.constate.done

```

21. ALGORITMO 20 — STREAMING COM CANCELAMENTO

```

import asyncio

def stream_with_cancel(client, prompt, cancel_event):
    """Stream with cancel"""
    async for text in stream_text(client, prompt):
        if cancel_event.is_set():
            await cancel_event.wait()
    return text

```

22. PROMPT LIBRARY — FUNDAMENTOS

22.1 Prompt de Sistema Canônico

```

[role: "system",
  [name: "Prompt", Postura: "Postura"],
  [context: "Contexto"],
  [estado: "Estado"],
  [resposta: "Resposta"]
]

```

22.2 Prompt de Tarefa

```
task: tarefa
input: entrada
output: resultado
description: descrição
format: formato
```

23. PROMPT LIBRARY — ESTRUTURADOS (JSON)

```
{
  role: "system",
  content: "Plano de conteúdo de curso de matemática"
}
```

24. PROMPT LIBRARY — CHAIN-OF-THOUGHT

```
task: tarefa
input: entrada
output: resultado
description: descrição
format: formato
steps:
  - decomponha o problema
  - identifique as variáveis
  - aplique a fórmula
```

25. PROMPT LIBRARY — FEW-SHOT

```
examples:
  - input: "A maçã é vermelha."
    output: "subject: color, property: color, polarity: positive"
  - input: "O céu é azul."
    output: "subject: sky, property: color, polarity: positive"
  - input: "A água é quente."
    output: "subject: water, property: temperature, polarity: positive"
```

Platform	Percentage of Respondents
WhatsApp	~85%
Telegram	~25%
Facebook	~15%

Category	Percentage
Estimated 10% to 15%	15%
Estimated 15% to 20%	20%
Estimated 20% to 25%	25%
Estimated 25% to 30%	30%
Estimated 30% to 35%	35%
Estimated 35% to 40%	40%

Category	Percentage
Design and Development	35%
Implementation and Deployment	25%
Testing and Quality Assurance	20%
Documentation and Project Management	10%
Client Communication and Support	10%

Social Media Platform	Percentage of Respondents
Facebook	68%
Instagram	42%
Twitter	35%
LinkedIn	28%
YouTube	25%
TikTok	22%
Snapchat	18%

29. PROMPT LIBRARY — RAG

Se a resposta não estiver no contexto, digite "Não sei com base nas fontes fornecidas".

Let's never chunk.

Example: they

ISSN 0005-707X

30. PROMPT LIBRARY – TOOLS

```
query db [str: str] => [str: str]
```

Para inovar, responda:

name: "bot", type: "args", value: "1"

31. PROMPT LIBRARY — SELF-CRITIQUE

• Esta corrección

Page 10 of 15

10. Tempestades e ondas

ES0005-1-0110 (date: 1/25/2015)

32. ALGORITMO FINAL — A WISDOM ENGINE (PREVIEW VOL. 3)

```
class WisdomEngine:
    def __init__(self, knowledge, ethics, history):
        self.k = knowledge
        self.e = ethics
        self.h = history
        self.principles = self.k.principles_for_decision_domain
        self.ethics_score = self.e.evaluate_decision
        self.precedents = self.h.find_similar_cases
        self.second_order = self.h.predict_ethics_based_on_precedent
        self.decision = self.k.decision
        self.ethics_score = self.e.evaluate_decision
        self.precedents = self.h.find_similar_cases
        self.second_order = self.h.predict_ethics_based_on_precedent
        self.recommendation = self.k.synthesize(self.principles, self.ethics_score, self.second_order)
```

No Vol. 3, detalhamos a Wisdom Engine completa com **autoconhecimento agêntico** e **autossabedoria determinística**.

33. GLOSSÁRIO

Algoritmo	Uso principal
Plan-and-Execute	Tarefas decompostas em etapas
Reflexion	Auto-correção iterativa

Algoritmo	Uso principal
Tree of Thoughts	Exploração de múltiplos caminhos
Self-Consistency	Voto de N amostras
ReWOO	Tools com placeholders
MRKL	Router + especialistas
BabyAGI	Lista infinita de tarefas
AutoGPT	Loop autônomo de alta agência
CAMEL	Role-play colaborativo
Voyager	Crescimento de skill library
DSPy	Prompting declarativo + otimização
Constitutional AI	Revisão por princípios
RLHF	Fine-tuning por feedback humano
Hybrid RAG	BM25 + dense + reranking
Agent Memory	Episódica + semântica
Self-Healing	Retry inteligente
Tool Router	Decision tree
Critic/Refiner	Loop de qualidade
State Machine	Orquestração cíclica
Streaming	Output progressivo + cancel

34. CALL TO ACTION — ECOSSISTEMA MMN_IA

Este ebook é o **Vol. 2** da coletânea **A IA Perfeita**.

- **Vol. 1** — Padrões & Arquiteturas (já publicado)

- **Vol. 2** — Códigos, Prompts & Algoritmos (este)
- **Vol. 3** — Skills, Agentic, Autocura & Sabedoria (em breve)

Plataforma MMN_IA: **oneverso.com.br**.

© 2026 Nexus HUB57 · Ecossistema MMN_IA · Todos os direitos reservados Versão 1.0.0 · Junho 2026 · Coleção A IA Perfeita, Vol. 2 de 3 — Códigos, Prompts & Algoritmos